



Gensyn Token Minting Cap

Security Assessment

September 17, 2026

Prepared for:

Harry Grieve

Gensyn

Prepared by: **Guillermo Larregay and Marcelo Morales**

Table of Contents

Table of Contents	1
Project Summary	2
Executive Summary	3
Project Goals	5
Project Targets	6
Project Coverage	7
Automated Testing	8
Summary of Findings	9
Detailed Findings	10
1. The bridged token reinitializer function lacks access control	10
2. The bridged token deployment script hard-codes one Safe address as the sole timelock proposer on every chain	12
A. Vulnerability Categories	14
B. Code Quality Recommendations	16
C. Test Suite Coverage Analysis	17
About Trail of Bits	20
Notices and Remarks	21

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Guillermo Larregay, Consultant
guillermo.larregay@trailofbits.com

Marcelo Morales, Consultant
marcelo.morales@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
August 16, 2026	Delivery of report draft
August 16, 2026	Report readout meeting
September 17, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

Gensyn engaged Trail of Bits to review the security of the Gensyn token minting cap.

The Gensyn token bridge enables the Gensyn token, an upgradeable ERC-20 token native to Gensyn's own network, to circulate on other chains through a mint-and-burn variant integrated with a LayerZero bridge adapter. When tokens are transferred off a given chain, the corresponding supply on that chain is burned; when tokens are transferred back to that chain, the corresponding supply is minted. Each bridged deployment is governed by a timelock contract, which holds administrative authority and grants the bridge adapter its minting and burning permissions.

The system comprises the token contracts and the deployment scripts responsible for instantiating them and configuring their governance. As these scripts constitute the sole mechanism for producing a live token deployment and its associated timelock, defects within them propagate directly into the resulting on-chain governance structure.

A team of two consultants conducted the review from August 7 to August 11, 2026, for a total of three engineer-days of effort. Our testing efforts focused on the bridged Gensyn token minting cap and the governance wiring produced by its deployment scripts. With full access to source code and documentation, we performed static and dynamic testing of the codebase.

Observations and Impact

The most notable issue (**TOB-GEN-CAP-2**) is that the bridged token's deployment script hard-codes a single Safe address as the timelock's sole proposer on every chain, with no admin role retained. On-chain checks show that address currently holds neither role on the live Ethereum timelock and has no code on several other chains that the token could be deployed to; therefore, running the script unchanged on any of these chains would produce a timelock that is unable to grant the roles its own bridge adapter needs.

We also identified an access-control gap in the `reinitializer` function (**TOB-GEN-CAP-1**): it is not role-gated, unlike the mint and burn functions beside it, though the live deployment has already consumed the relevant initializer version.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that Gensyn take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.

- **Strengthen government and deployment controls.** Restrict deployment scripts to an explicit allowlist of target chains, add validation to ensure that any hard-coded proposer or admin address holds code and the intended role on that chain, and gate sensitive functions.
- **Improve the test suite.** Add coverage for bridged token burning, mint and burn function return values, reinitialization after minting, and independent EIP-712 domain construction. Rerun the mutation campaign after these changes. See [appendix C](#) for details.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	0
Medium	0
Low	1
Informational	1
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Access Controls	1
Configuration	1

Project Goals

The engagement was scoped to provide a security assessment of the Gensyn token minting cap. Specifically, we sought to answer the following non-exhaustive list of questions:

- How does the canonical Gensyn token enforce its minting cap?
- Can any initialization, minting, or upgrade path bypass or weaken the cap?
- Which privileged actors can mint tokens, assign minting permissions, or modify the implementation?
- How do token burns affect the remaining minting capacity and maximum supply?
- Does each bridged-token deployment enforce an independent cap, or are caps synchronized across networks?
- Can the aggregate bridged-token supply exceed the canonical token's minted or circulating supply?
- What bridge and access-control assumptions ensure that bridged tokens are minted only after corresponding tokens are locked or burned?
- Does the test suite adequately verify cap enforcement, boundary conditions, bridge accounting, initialization sequences, and upgrade invariants?

Project Targets

The engagement involved reviewing and testing the following target.

private-smart-contracts

Repository	https://github.com/gensyn-ai/private-smart-contracts/
Version	Commit 0f896fcaab17ceb9424eef016a60a9fd5d761ff1
Type	Solidity
Platform	EVM

The following is the public mirror of the private repository, provided by Gensyn at the conclusion of the engagement:

Public mirror

Repository	https://github.com/gensyn-network/gensyn-token/
Version	Commit b46752c
Type	Solidity
Platform	EVM

The following are the SHA-256 hashes of the in-scope files:

- `src/BridgedGensynToken.sol`
 - `8cae74d3b219571698a93ab3e5c56c8a38a1b2f295326cece861de67ebbb51de`
- `src/BridgedGensynTokenV2.sol`
 - `6e2f048b22e38e51f36b9317084f7ea2a9e49b965be026ea47340ebcd140836c`
- `src/GensynToken.sol`
 - `894c8888d7bdde9ae53bd98f47c3f2ced40073c7cd44eea8c63c726c29b7f516`
- `src/Utils/Utils.sol`
 - `4d97dac1d9b64d3bf6d4ae22caac51b9222ebc579d559b40ed71004abfde4464`

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- A manual review of cap enforcement on token implementations, including initialization, reinitialization, minting, and burning.
- An overview of the cross-chain token architecture, including the LayerZero mint-and-burn adapter, canonical and bridged token supply relationships, bridge message flows, and cap assumptions.
- A review of privileged operations and trust boundaries, including those related to the minter and burner roles, looking for opportunities for privileged actors to modify or bypass supply restrictions.
- An adversarial analysis of cap-related invariants and edge cases, including minting at and around the cap, burning and re-minting, initialization sequencing, and deploying multiple bridged tokens.
- Execution and assessment of the existing unit and fuzz tests, and implementation and analysis of a mutation-testing campaign.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- The review focused on minting-cap enforcement in GensynToken and BridgedGensynToken. We reviewed deployment scripts, tests, and configuration as supporting evidence. Third-party libraries and dependencies were out of scope, although we reviewed how the in-scope contracts use and interact with them.
- We examined the LayerZero adapter only where its behavior affects token supply. We did not cover the LayerZero protocol or its components.
- Testing relied on the implementations, local test environment, and deployment configuration available during the engagement.

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Test Harness Configuration

We used the following tools in the automated testing phase of this project:

Tool	Description	Policy
mewt	Mewt is a mutation testing tool. Mutation testing works by making small changes to source code and then running the test suite against each change.	Appendix C

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	The bridged token reinitializer function lacks access control	Access Controls	Informational
2	The bridged token deployment script hard-codes one Safe address as the sole timelock proposer on every chain	Configuration	Low

Detailed Findings

1. The bridged token reinitializer function lacks access control

Severity: Informational

Difficulty: Not Applicable

Type: Access Controls

Finding ID: TOB-GEN-CAP-1

Target: token/src/BridgedGensynToken.sol

Description

The reinitializer function, which destroys the bridged token's initial supply, carries no caller authorization, while the two supply-changing functions beside it are both role-gated.

```
function reinitialize(address _recipient) external reinitializer(2) {
    // Burn initial supply from recipient
    _burn(_recipient, INITIAL_SUPPLY);

    // Ensure total supply is zero after upgrade
    /// @dev This is the bridged token, so its supply should start at 0
    assert(totalSupply() == 0);
}

/// @inheritdoc IMintableBurnable
function mint(address _to, uint256 _amount) external onlyRole(MINTER_ROLE) returns
(bool success) {
    _mint(_to, _amount);
    success = true;
}
```

Figure 1.1: The reinitializer function accepts any caller and any recipient, unlike the role-gated mint function beside it. (token/src/BridgedGensynToken.sol#L28-L41)

OpenZeppelin's reinitializer modifier enforces only the initialization version and performs no caller-authorization check. Consequently, any account can call reinitialize on a proxy that has not yet consumed version two. The call succeeds only if _recipient holds INITIAL_SUPPLY, and burning that amount reduces totalSupply() to zero; otherwise, _burn or the trailing assertion reverts and rolls back the version change. In the deployment produced by the bridged-token script, initialize assigns the entire initial supply to 0x000...dEaD. An attacker can therefore successfully call reinitialize only by specifying that address, which produces the same zero-supply state intended by the deployer. The defect becomes exploitable if a future upgrade points a compatible proxy at this

implementation, while version two remains unused and the entire initial supply is held by a live address: any account could specify that address and irreversibly burn its tokens.

Exploit Scenario

Gensyn adds a bridge minting and burning capability to an existing canonical GensynToken proxy by upgrading it to BridgedGensynToken, a step the deployment plan does not describe but the contract hierarchy permits and the timelock can authorize. That proxy minted its full supply to a live treasury and never consumed version two, so the upgrade alone arms `reinitialize` against a live balance. Eve observes it and calls `reinitialize` in the same block, naming the treasury as the recipient; the version check passes, the burn succeeds because the treasury holds the entire supply, and the assertion passes because the supply is now zero. No role gates the call, and the timelock that authorized the upgrade does not govern it, so one unreviewed operational step places the whole supply within reach of any account.

Recommendations

Short term, restrict `reinitialize` to `DEFAULT_ADMIN_ROLE` and update the deployment script to match, since it calls the function from the broadcasting account while that role is granted to the timelock. This will address the issue because the burn can then occur only at the direction of the account the token recognizes as its administrator.

Long term, fold the supply reduction into `initialize` so that no second initialization version is ever exposed on a live proxy. This will prevent the issue from recurring because an unconsumed initializer version will stop being a standing capability that any future implementation upgrade can arm.

2. The bridged token deployment script hard-codes one Safe address as the sole timelock proposer on every chain

Severity: Low

Difficulty: High

Type: Configuration

Finding ID: TOB-GEN-CAP-2

Target: token/script/BridgedGensynToken.s.sol

Description

The bridged token deployment script has no chain ID validation and names a hard-coded address as the timelock's sole proposer and canceller; it does not set a timelock admin. It therefore produces identical governance wiring wherever it runs, unlike the canonical deployment script, which restricts itself to two Gensyn chain IDs.

Because `TimelockController` grants `DEFAULT_ADMIN_ROLE` only to itself when the admin argument is zero, deploying to a chain where that address cannot originate transactions yields a timelock whose proposer set cannot be repaired through its own role machinery: `MINTER_ROLE` and `BURNER_ROLE` never reach the bridge adapter, supply stays frozen at zero, and `_authorizeUpgrade` is unsatisfiable until the entire stack is redeployed.

That address is a three-of-five Safe address on the Ethereum mainnet holding neither `PROPOSER_ROLE` nor `CANCELLER_ROLE` on the live bridged timelock, and it has no code at all on Base, Arbitrum One, Optimism, or BSC. Therefore, the constant is not merely chain-dependent but also stale: production has already rotated governance away from it, so the next run of this script names a retired proposer on Ethereum as readily as on a chain the token has never reached.

```
uint256 timelockMinDelay = 7 days;

address gensynSafeAddress = 0x90442673dae1b1572a3D994A4D795c8977A97ECD;

address[] memory timelockProposers = new address[](1);
timelockProposers[0] = gensynSafeAddress;

address[] memory timelockExecutors = new address[](1);
timelockExecutors[0] = address(0); // allow anyone to execute

address timelockAdmin = address(0); // renounce admin role to prevent centralization

address gensynTokenRecipient = 0x00000000000000000000000000000000dEaD;
```

```
// Deploy Timelock
timelock = new TimelockController({
    minDelay: timelockMinDelay, proposers: timelockProposers, executors:
timelockExecutors, admin: timelockAdmin
});
```

*Figure 2.1: The chain-independent proposer constant and the renounced timelock admin
([token/script/BridgedGensynToken.s.sol#L22-L39](#))*

Exploit Scenario

An operator extends the bridge to a new chain and runs the script unchanged; the deployment succeeds and reports no error. Days later, the team attempts to grant MINTER_ROLE and BURNER_ROLE to the newly deployed MintBurnOFTAdapter but cannot: scheduling that grant requires PROPOSER_ROLE, a role that the script assigned to an address where no account exists on this chain, and that no other account holds. The timelock is its own only administrator, so no transaction can add one. The token, its timelock, and their block explorer verifications must all be discarded and redeployed.

Recommendations

Short term, restrict the script to an explicit allowlist of the chain IDs the bridged token is intended for, and update the script to read the timelock proposer from a per-chain source validated to be nonzero and to hold code on the target chain. This will address the issue because the script will then refuse to run on any chain whose proposer has not been established, and will refuse to name a proposer that cannot originate transactions.

Long term, grant CANCELLER_ROLE and PROPOSER_ROLE to different parties and provision an explicit recovery administrator for every timelock the repository deploys. This will prevent the issue because OpenZeppelin will grant each proposer CANCELLER_ROLE as well, so adding a second proposer alone will leave a lost or hostile first proposer able to cancel any repair.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Recommendations

This appendix contains recommendations for findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Add validation of existing supply during initialization of the V2 cap.** `reinitializeV2()` should revert when `totalSupply()` already exceeds `INITIAL_SUPPLY`; otherwise, an over-cap V1 deployment would remain over the cap after migration.
- **Consider replacing the bridged token's mint-then-burn initialization with a zero-supply initializer.** Reusing the canonical initializer temporarily mints and then burns 10 billion tokens, increasing gas costs, emitting misleading events, and requiring a fragile second initialization call.
- **Make bridged token deployments install and initialize the capped implementation atomically.** The current workflow deploys `BridgedGensynToken` uncapped and does not configure the `LayerZero` adapter or its mint and burn roles.
- **Standardize deployment-script names and entry points.** Both scripts use the same contract name, while the deployment command calls `deploy()` and the bridged script only implements `run()`, causing the documented workflow to fail.
- **Add validation of all deployment and initialization parameters.** Reject zero-address administrators, recipients, proposers, and adapter addresses, and verify the selected chain before deployment to prevent unusable or incorrectly configured contracts.
- **Document the cap's exact meaning and bridge trust assumptions.** The cap limits the current supply on one chain, not global cross-chain supply.
- **Add UUPS storage-layout and upgrade-invariant checks.** Tests should verify storage compatibility and ensure every proposed implementation preserves critical properties such as cap enforcement.

C. Test Suite Coverage Analysis

Line coverage measures the lines of code that were tested; it does not indicate whether an incorrectly implemented line would cause a test to fail. Mutation testing measures the latter. A mutation test makes many small, deliberate edits, called mutants, to the source (such as changing `>` to `>=`, deleting a validation check, or flipping `+` to `-`), and then runs the whole test suite against each mutant and records the outcome:

- A mutant is **caught** if at least one test fails. The suite notices the change.
- A mutant is **uncaught (it survived)** if every test passes. Nothing in the suite detects the change. This is a genuine gap, unless the edit happens to be equivalent (for example, replacing `x == 0` with `x <= 0` for unsigned types).
- A mutant is **skipped** if it is not tested because a more severe mutant on the same line already survived.

The mutation testing campaign we ran on the Gensyn token minting cap codebase resulted in 70 mutants. Of the 58 tested, 50 were caught and 8 survived; the remaining 12 were skipped. All six generated mutants for `BridgedGensynTokenV2` were caught, and the tests detected most of the generated changes to the core ERC-20 token code and access-control paths. Of the 8 survivors, 5 expose production-contract test gaps and 3 belong to non-production deploy/test scaffolding that should be excluded from the production mutation results.

Coverage Gaps

This section summarizes the coverage gaps revealed by the mutation testing campaign.

The bridge mint and burn functions are under-verified

`BridgedGensynToken` exposes role-gated mint and burn functions for the cross-chain adapter. No test calls the two-argument function for burning bridged tokens. Existing burn tests cover only the inherited self-burn function. Consequently, a mutation that makes calls to the bridge token burning function always revert survives, leaving the cross-chain supply-destruction path unverified.

Tests exercise state changes after the mint function but do not check its Boolean return value, allowing a mutation that causes the function to return false after a successful mint to survive. Tests should verify the roles, balance and supply changes, and true return values of both bridge functions.

reinitialize is not tested after prior bridge minting

`BridgedGensynToken.reinitialize` burns `INITIAL_SUPPLY` and asserts that `totalSupply` is zero, but existing tests invoke it immediately after `initialize`. Because

an administrator can grant `MINTER_ROLE` and call the mint function first, the assertion is not redundant: the original implementation then reverts, while a mutant without the assertion completes with positive supply. A sequencing test should cover this case.

The permit test validates a signature against the contract it is testing

The permit test reads `DOMAIN_SEPARATOR` from `GensynToken`, signs with it, and submits the signature to the same contract. If `__ERC20Permit_init` is skipped, the EIP-712 name and version remain empty, yet the test still passes because signing and verification use the same incorrect domain. Clients signing with the intended Gensyn/version 1 domain would fail. The test should independently compute and validate the expected separator from the literal name, version, chain ID, and proxy address.

Non-production campaign scope

The remaining three survivors are in `Utils.sol`, which is deployment/test scaffolding rather than a production token contract. They are excluded from the production-contract findings and should be removed from the production mutation campaign scope.

Recommendations

Based on the results of the mutation testing campaign, we recommend that the Gensyn team take the following actions, listed in order of priority.

Priority 0: Cover bridge behavior and initialization invariants

- **Test the role-gated burn function.** Write a test to grant `BURNER_ROLE`, mint a known balance, burn a fuzzed amount, and assert that the target's balance and total supply each decrease by that amount, that the call returns true, and that a caller without the role is rejected. This will close the campaign's most significant gap.
- **Test `reinitialize` after prior minting.** Write a test to grant `MINTER_ROLE`, mint a positive amount before initialization version 2, and assert that `reinitialize` reverts instead of completing with a nonzero supply. This test should verify the post-condition enforced by the internal assertion.

Priority 1: Verify integration and interface assumptions

- **Add a test to check the EIP-712 domain against an expected value.** The test should compute the expected domain separator independently from the literal name Gensyn, version 1, chain ID, and token address, and assert that `DOMAIN_SEPARATOR` equals it. At least one permit test should sign against that independently computed domain rather than reading it from the contract, so a missing or incorrect domain initialization fails the test.
- **Add or update tests to assert the mint and burn functions return success.** Existing mint call sites should check that the returned flag is true, and the new burn test should do the same. This will pin the documented `IMintableBurnable`

interface semantics, but it is a lower priority because the current adapter ignores the flag and relies on reverts.

Priority 2: Scope and complete the campaign

- **Separate deployment/test scaffolding from the production files.** Consider moving `Utils.sol` out of `src/` and into `test/` or `script/` so that its purpose is clear from the project layout.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Gensyn under the terms of the project statement of work and has been made public at Gensyn's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.